

WiFi encrypted traffic classification

Mattia Fiore – Pasquale Lobaccaro

Step 1: Capturing data

For the first step we used 3 different devices:

- A Linux (Ubuntu) system to perform the capturing,
- A Mobile phone which provided the internet connection (iOS),
- A MacOS device that consumed the content.

We captured 8 different application traffic:

- Netflix
- Youtube
- Whatsapp web
- Telegram web
- Instagram web
- Zoom
- Amazon
- Wikipedia

To perform the capturing we first need to set the monitor mode and the specific channel to monitor.

```
:~$ sudo ifconfig wlp1s0 down
:~$ sudo iwconfig wlp1s0 mode monitor
:~$ sudo ifconfig wlp1s0 up
:~$ sudo iwconfig wlp1s0 channel 6
```

To see the list of possible channels we can use the following command:

```
mattiafiore@mattiafiore-CREM-WXX9:~$ iwlist wlp1s0 channel
wlp1s0    32 channels in total; available frequencies :
Channel 01 : 2.412 GHz
Channel 02 : 2.417 GHz
Channel 03 : 2.422 GHz
Channel 04 : 2.427 GHz
Channel 05 : 2.432 GHz
Channel 06 : 2.437 GHz
Channel 07 : 2.442 GHz
Channel 08 : 2.447 GHz
Channel 09 : 2.452 GHz
Channel 10 : 2.457 GHz
Channel 11 : 2.462 GHz
Channel 12 : 2.467 GHz
Channel 13 : 2.472 GHz
Channel 14 : 2.484 GHz
Channel 36 : 5.18 GHz
Channel 40 : 5.2 GHz
Channel 44 : 5.22 GHz
Channel 48 : 5.24 GHz
Channel 52 : 5.26 GHz
Channel 56 : 5.28 GHz
Channel 60 : 5.3 GHz
Channel 64 : 5.32 GHz
Channel 100 : 5.5 GHz
Channel 104 : 5.52 GHz
Channel 108 : 5.54 GHz
Channel 112 : 5.56 GHz
Channel 116 : 5.58 GHz
Channel 120 : 5.6 GHz
Channel 124 : 5.62 GHz
Channel 128 : 5.64 GHz
Channel 132 : 5.66 GHz
Channel 136 : 5.68 GHz
Current Frequency:2.437 GHz (Channel 6)
```

To start the capturing we used tcpdump:

```
$ sudo tcpdump -e -v -w name_of_the_file.pcap
```

The flag used are:

- -e flag: which is used to get information about source and destination MAC addresses,
- -w flag: used to write the information in a file,
- -v flag: used to control verbosity.

The capturing was repeated for every type of traffic. All the files generated were stored in the folder: 'PCAP'

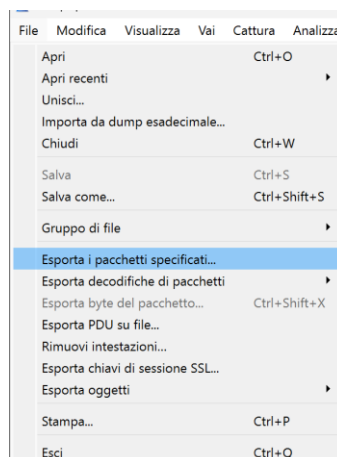
Step 2: Filtering the .pcap

The capturing was done in a very isolated place so there were just a few packets which were not from the interested traffic. Nonetheless we used Wireshark to filter out the .pcap file.

First using the following filter:

```
wlan.da == 14:7d:da:93:75:71 or wlan.sa == 14:7d:da:93:75:71
```

Then using a Wireshark function, we were able to create a separate .pcap file with just the interesting packets:



All the files generated were stored in the folder: 'FILTERED PCAP'. Files can be found at the following link: <https://drive.google.com/drive/folders/1aGjX3T2ytm9zSOHZETLdRBGOIUlxWV5Y?usp=sharing>.

Step 3: Converting .pcap into .csv

To convert .pcap file into .csv file we used a line terminal command for each of the file:

```
tshark -r "name_of_the_pcap_file.pcap" -2 -T fields -e frame.number -e frame.time -e wlan.sa -e wlan.da -e wlan.ssid -e data.len -E header = y -E separator=, -E quote=d -E occurrence=f > "name_of_the_csv_file.csv"
```

All the files generated were stored in the folder: 'FILTERED CSV'

Step 4: Preparing the data with Python

Using python and the Pandas library we prepared the data to be fed into the machine learning algorithm. We computed the following features:

- Average data length
- Standard deviation data length
- Average interarrival time
- Standard deviation interarrival time

- Number of packets

This was done both in uplink and downlink. With the notebook we created different datasets based on the interval for which the statistics were computed.

We used the following intervals: 0.15s, 0.30s, 0.5s, 1.0s, 2.5s.

All the files were stored in the folder: 'DATASETS'.

Notebook

We imported the following libraries:

```
import pandas as pd
import numpy as np
from datetime import datetime
import re
from statistics import mean, stdev
```

We defined a list with all the names of the files on which we will iterate:

```
file_names = ['amazon_filtered.csv',
              'instagram_filtered.csv',
              'netflix_filtered.csv',
              'telegram_filtered.csv',
              'whatsappweb_filtered.csv',
              'wikipedia_filtered.csv',
              'youtube_filtered.csv',
              'zoom_filtered.csv']
```

Then we repeat the following script for all the elements in the list:

```
for file_ in file_names:

    # we read the csv file
    df = pd.read_csv(file_)

    # we drop columns that we will not need
    df = df.drop(columns=['frame.number', 'wlan_radio.signal_dbm', 'wlan.ssid'])
```

We had to convert the time format since it was in the following form:

"Aug 10, 2023 12:57:44.935991000 CEST"

To do that we used Regular Expressions. With the 're' library function 'findall' and with the regex "\d+" we were able to extract all the numbers in the string:

text = "Aug 10, 2023 12:57:44.935991000 CEST"

re.findall(r'\d+', text) -> [10, 2023, 12, 57, 44, 935991000]

Then we created a different DataFrame with all the time information. With the function:

pd.to_datetime(dataframe)

We create the datetime object.

```
# Using regex we create a list with just the numbers
df['list'] = df['frame.time'].apply(lambda x: re.findall(r"\d+", x))

# We create a second dataframe with the extracted time information
df2 = pd.DataFrame(df['list'].tolist(), columns=['day', 'year', 'hour', 'minute', 'second', 'microsecond'])

# We add the month
# Note: we cleared the month, which was written Aug, with the regex
df2['month'] = 8

# We create a datetime column on the original dataframe
df['datetime'] = pd.to_datetime(df2[['year', 'month', 'day', 'hour', 'minute', 'second', 'microsecond']])

# We drop columns that we will not need
df = df.drop(columns=['frame.time', 'list'])
```

After converting the string into a proper datetime object we compute the timestamp. We also deleted all the columns used to do the conversion.

```
# We drop rows without data.len parameters
df.dropna(inplace=True, subset = ['data.len'])

# we convert the datetime into the amount of seconds
df['timestamp'] = df['datetime'].apply(lambda x: (x - df['datetime'].iloc[0]).total_seconds())

# we remove the datetime column
df = df.drop(columns=['datetime'])
```

Then we divided the two different directions of the traffic and we created a clean dataset on which we'll put all the statistics.

```
uplink_df = df[(df['wlan.sa']=='14:7d:da:93:75:71') & (df['wlan.da'] == '02:7d:60:f0:2d:64')]
downlink_df = df[(df['wlan.sa']=='02:7d:60:f0:2d:64') & (df['wlan.da'] == '14:7d:da:93:75:71')]
```

```
# We create the new DataFrame on which we will put the cleaned data
columns = ['avg datalen dl',
           'std datalen dl',
           'n_packets dl',
           'avg intertime dl',
           'std intertime dl',
           'avg datalen ul',
           'std datalen ul',
           'n_packets ul',
           'avg intertime ul',
           'std intertime ul']

clean_dataset = pd.DataFrame(columns = columns)
```

We create a loop using the numpy.arange:

```
# We define parameters for the arange
max_time = downlink_df.iloc[-1]['timestamp']

# TIME INTERVAL
step = 2.5

for i in np.arange(step, max_time, step):
```

Inside the loop we computed the features present in the list 'columns'. Some control was added since the library does not allow to compute the mean on a single element or the standard deviation with two elements. We set the value of the mean to 0 if no packets were captured. We set the value of the standard deviation to -1 if less than two packets were captured.

```
# create partition for downlink
dl = downlink_df[(downlink_df['timestamp'] >= i-step) & (downlink_df['timestamp'] < i)]
# create partition for uplink
ul = uplink_df[(uplink_df['timestamp'] > i-step) & (uplink_df['timestamp'] < i)]

#avg data_len downlink
dl_mean = dl['data.len'].mean()

#std data_len downlink
dl_std = dl['data.len'].std()

#number of packets downlink
dl_n = dl.shape[0]

# computing interarrival times downlink
dl_int_times = []
for j in range(dl['timestamp'].shape[0]-1):
    previous = dl['timestamp'].iloc[j]
    successive = dl['timestamp'].iloc[j+1]
    dl_int_times.append(successive-previous)

if len(dl_int_times) != 0:
    #avg downlink interarrival time
    dl_int_mean = mean(dl_int_times)

    #std downlink interarrival time
    if len(dl_int_times) >= 2:
        dl_int_std = stdev(dl_int_times)
    else:
        # stdev cannot be computed with less then two elements
        dl_int_std = -1
else:
    # mean cannot be computed with 0 elements
    dl_int_mean = 0
```

The same code is repeated for the uplink. Then we added the computed statistics for each interval in the clean dataset:

```
# Create the row
row = [dl_mean, dl_std, dl_n, dl_int_mean, dl_int_std, ul_mean, ul_std, ul_n, ul_int_mean, ul_int_std ]
# add the new row
clean_dataset.loc[index] = row
index += 1
```

Since the files were named 'nameoftheapplication_filtered.csv' we used the regex '[a-z]*' so to get the string 'nameoftheapplication'. Then we saved the cleaned datasets on a different file.

```
# we add the supervised label
name = re.findall(r'[a-z]*', file_)[0]
clean_dataset['supervised'] = name

# we save the dataset with a new name
clean_dataset.to_csv(name + '.csv')
```

At last, we computed a unified dataset with all the cleaned datasets.

```
names = ['amazon', 'instagram', 'netflix', 'zoom', 'telegram', 'whatsappweb', 'youtube', 'wikipedia']

frames = [pd.read_csv(i+'.csv') for i in names]

dataset = pd.concat(frames)

dataset.to_csv('dataset.csv')
```

The notebook can be found at the following link:

<https://colab.research.google.com/drive/1cj6RYHOImz1qZL5ftn5HZ9SeD4AJxRpw?usp=sharing>

Step 5: Feeding the data into the machine learning algorithm

We chose the K-nearest neighbours for the purpose, and we developed a notebook on which we were able to try many different combinations to find a reasonably good accuracy.

We tried with all the datasets based on the interval duration: 0.15s, 0.30s, 0.50s, 1.00s, 2.50s.

The classification was done in two different ways:

- all the different classes,
- grouped classes.

Video on Demand	Netflix, Youtube
Messaging	Whatsapp web, Telegram web
Video Conferencing	Zoom
Web Browsing	Amazon, Wikipedia
Social Network	Instagram

The classes were grouped in the following way:

Notebook

First, we imported the libraries.

```
[10] import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
```

Then we set the parameters:

```
files = ['dataset-0.15.csv',
        'dataset-0.3.csv',
        'dataset-0.5.csv',
        'dataset-1.0.csv',
        'dataset-2.5.csv']

# All possible features:
# 'avg datalen dl'
# 'std datalen dl'
# 'n_packets dl',
# 'avg intertime dl',
# 'std intertime dl',
# 'avg datalen ul',
# 'std datalen ul',
# 'n_packets ul',
# 'avg intertime ul',
# 'std intertime ul'

# Here it is possible to select different features
features = ['avg datalen dl', 'std datalen dl', 'std datalen ul', 'avg datalen ul']

# Here it is possible to modify the metric
metric = 'euclidean'

# Here you can set the maximum k for the test
K_max = 50
```

In the notebook there are two sections:

- The testing section,
- The result section.

The testing section was used to try many different parameters such as: features, metric, dimension of k. The test of the algorithm was done both on the grouped and the non-grouped dataset for all the possible datasets in the list 'files'.

```
for l in files:

    # read the csv file
    df = pd.read_csv(l)

    # These lines are used to get rid of the rows with 0 and -1
    # since both mean and std are computed when packets are more than 2
    df.drop(df[df['n_packets d1'] < 2].index, inplace = True)
    df.drop(df[df['n_packets u1'] < 2].index, inplace = True)

    # select the features
    df = df[features + ['supervised']]
    df.dropna(inplace = True, subset= features)

    #randomly scramble the dataset
    df = df.sample(frac=1)

    #creating the grouped dataset
    df2 = df.copy()

    dictionary = {
        'netflix': 'VOD',
        'youtube': 'VOD',
        'amazon': 'Web browsing',
        'wikipedia': 'Web browsing',
        'instagram': 'Social Network',
        'whatsappweb': 'Messaging',
        'telegram': 'Messaging',
        'zoom': 'Video call'
    }

    df2['supervised'] = df['supervised'].apply(lambda x: dictionary[x])
```

We tested various values of K using a cross-validation with 10 folds as used in '*Class of Service Mapping for QoS: A Statistical Signature based Approach to IP Traffic Classification*'.

```
X = df.drop(['supervised'],axis=1);
Y = df['supervised'];

ACCURACY = []
# We repeat for all K up to K_max
for k in range(1,K_max):
    knn = KNeighborsClassifier(n_neighbors=k, weights='distance', metric = metric)
    # we use cross-validation 10 fold
    score = cross_val_score(knn, X, Y, cv = 10)
    # we add to the accuracy list the mean of the cross-validations scores
    ACCURACY.append(np.mean(score))
```

We did the same for the df2. After some testing we found some good parameters for the algorithm and in the result section we repeated the code just for one dataset and we ran the algorithm on the best value of k. The best value of k was computed with a cross-validation 10-fold.

The results are computed without cross-validation with the train_test_split function with test_size 10%.

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.1)
```

We then plotted the confusion matrix associated to the best results and a x-y plot of the accuracy depending on the dimension of K.

```
# computing best k
bestk = np.argmax(ACCURACY)+1
# Re running the algorithm
knn = KNeighborsClassifier(n_neighbors=bestk, weights='distance')
knn.fit(X_train, y_train)
# Compute predictions
knn_predict = knn.predict(X_test)
# Plot the confusion matrix
fig, ax = plt.subplots(figsize=(15, 15));
ConfusionMatrixDisplay.from_predictions(knn_predict, y_test, ax=ax, normalize='true');

fig = plt.figure();
plt.plot(range(1,K_max),ACCURACY);
plt.grid(visible=True);
plt.xlabel('K');
plt.ylabel('Accuracy');
```

All the code can be found at the following link:

<https://colab.research.google.com/drive/1L0Bc7GDB16GZ6xnTiLzGAI33Q-hKwWL4?usp=sharing>

Step 6: Results

We found that the best performing dataset was the one with the time intervals of 0.15 seconds and that the performance was decreasing the bigger the time interval was. This may be credited to the decrease of the size of the datasets. Since the starting dataset was the same, increasing the time interval reduces the data that is fed to the algorithm and this may cause the reduction in performance.

We tested the following metrics: 'minkosky', 'sqeuclidean', 'cosine' and 'euclidean'. We found that the best performing metric was 'euclidean'.

We also found that combining features of the uplink and downlink for training the machine learning algorithm gives better performance with respect to using just uplink or downlink features.

In the following pages we reported the most interesting examples.

#1

Type: Grouped

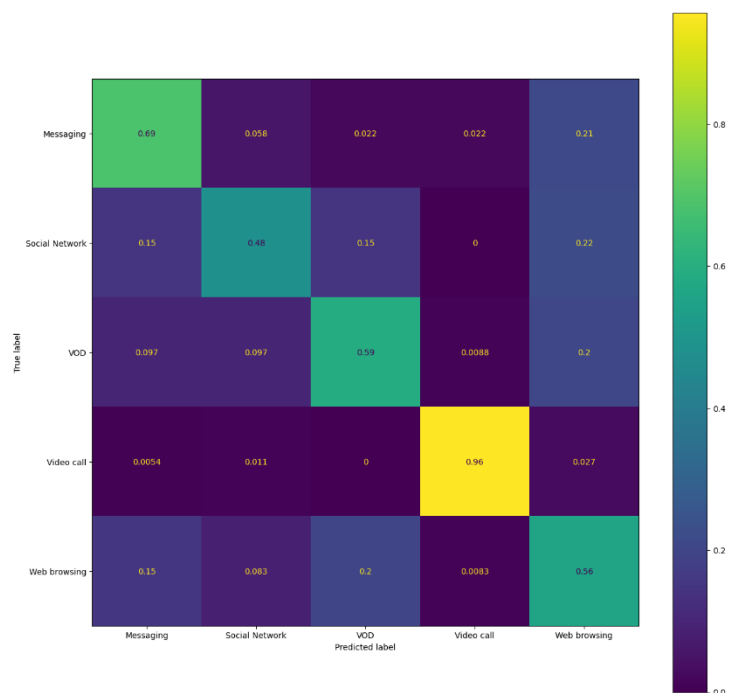
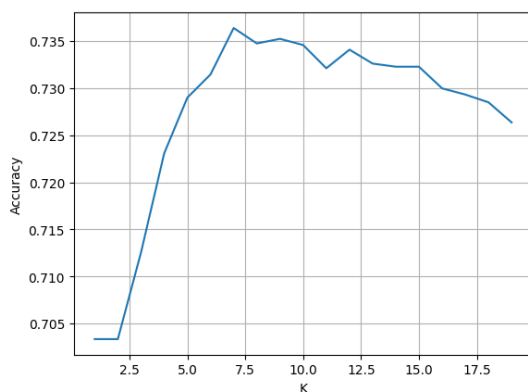
Dataset: 0.15 seconds

Metric: Euclidean

Feature used: all

Best accuracy: 73.6%

Best K: 7



The classification shows very good results in classifying the video conferencing traffic. This was an expected result since video conferencing has small packets and has a much bigger traffic in the uplink direction with respect to the other traffic classes.

#2

Type: ungrouped

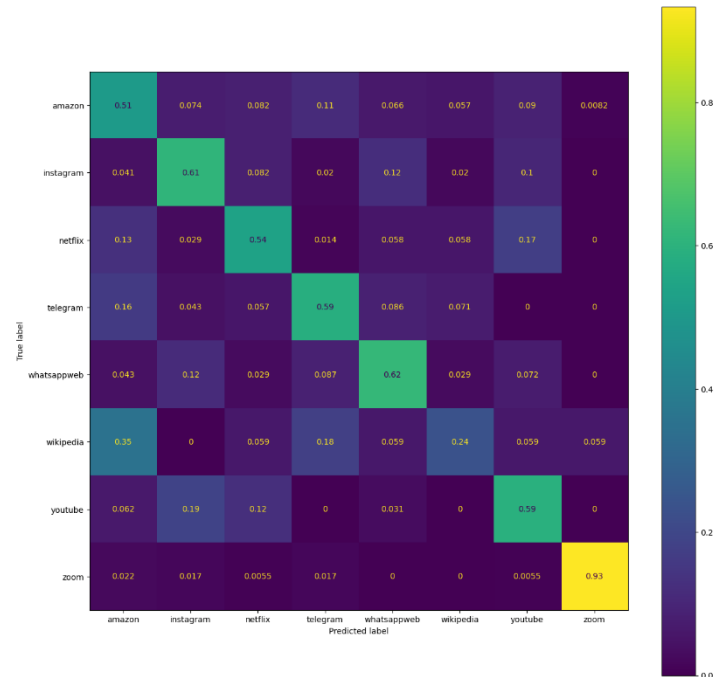
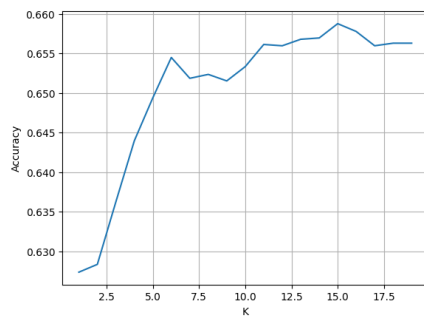
Dataset: 0.15 seconds

Metric: Euclidean

Feature uses: all

Best accuracy: 65.8%

Best K: 15



In the classification of all the classes of traffic we find that 'zoom' is still classified correctly most of the times and that was expected. Instead, wikipedia gets a very low score.

For this reason we also tried the same algorithm on the same dataset but without 'amazon' and 'wikipedia' classes.

#3

Type: Grouped without amazon and wikipedia

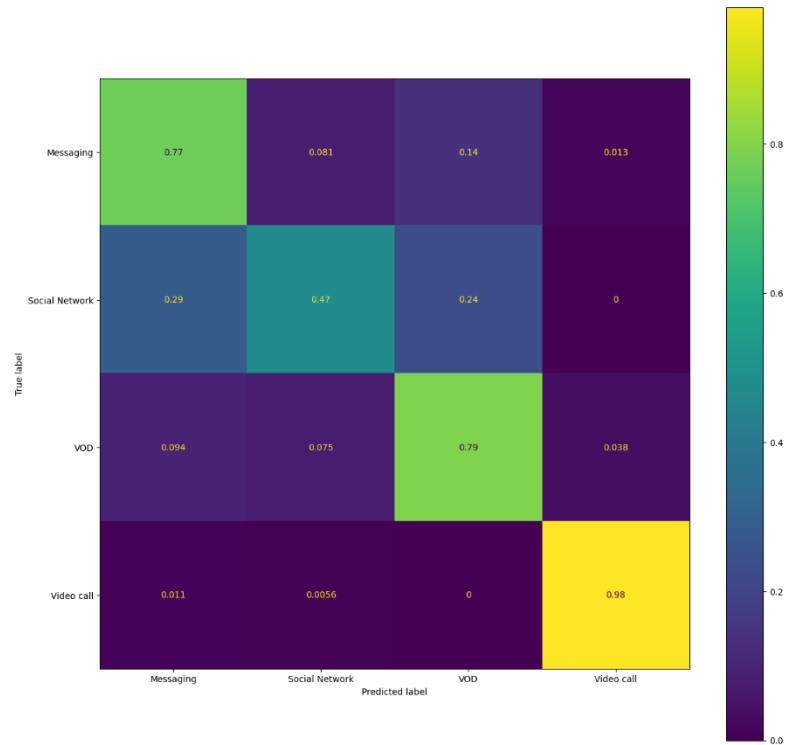
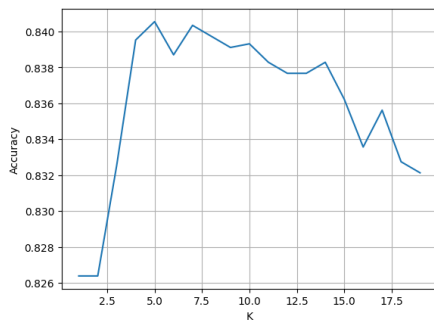
Dataset: 0.15 seconds

Metric: Euclidean

Feature uses: all

Best accuracy: 84.05%

Best K: 5



Overall performance of the algorithm without the 'amazon' and 'wikipedia' data is improved substantially.

#4

Type: ungrouped without amazon and wikipedia

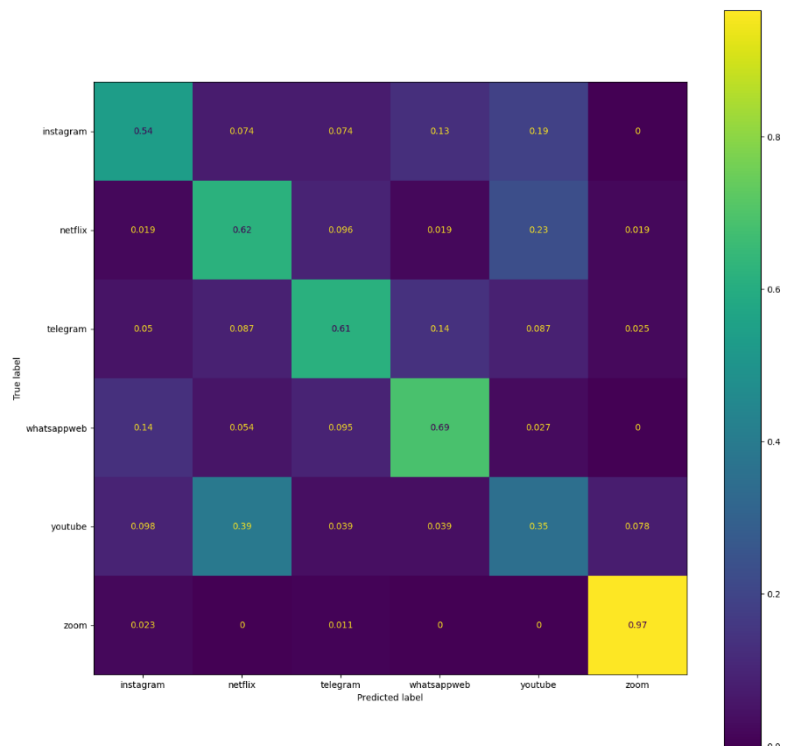
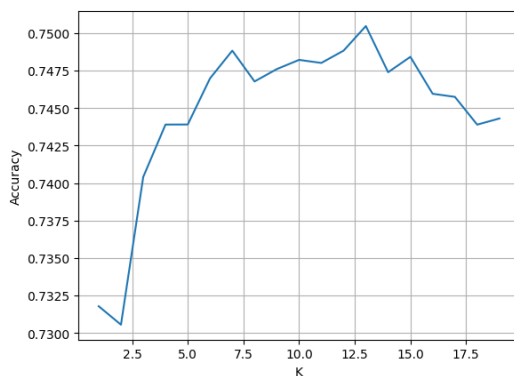
Dataset: 0.15 seconds

Metric: Euclidean

Feature uses: all

Best accuracy: 75.3%

Best K: 9



#5

Type: grouped

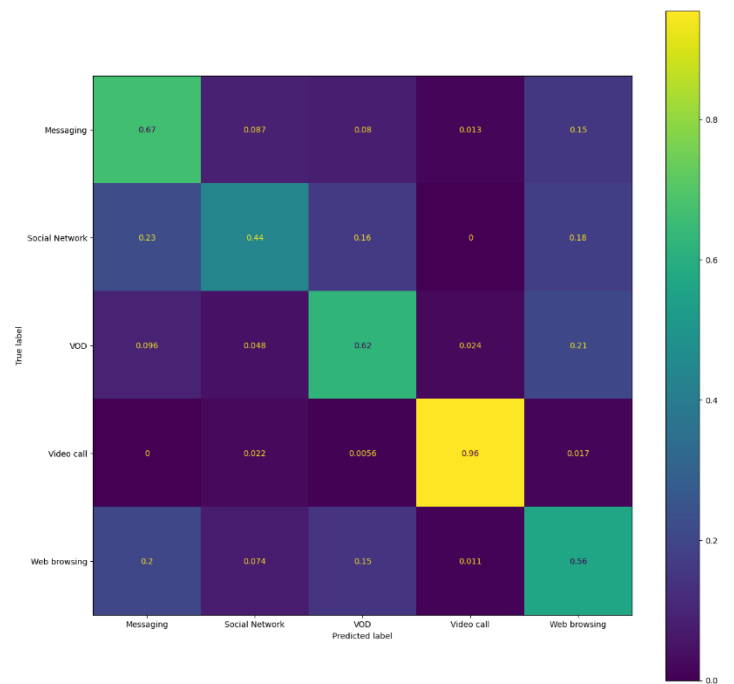
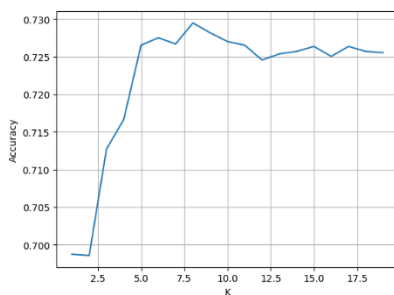
Dataset: 0.15 seconds

Metric: Euclidean

Feature used: average data length uplink, average data length downlink, standard deviation data length uplink, standard deviation data length downlink.

Best accuracy: 72.9%

Best K: 8



This is a very nice result since we get similar performance to the first example but using just 4 features instead of 10.

#6

Type: ungrouped

Dataset: 0.15 seconds

Metric: Euclidean

Feature used: average data length uplink, average data length downlink, standard deviation data length uplink, standard deviation data length downlink.

Best accuracy: 65.18%

Best K: 19

